

Hardware Software Interface

pinCrack Application Report

1. Hardware Specification

- 1 Raspberry Pi 3 Model B
- 2 LED's (Red and Green)
- 1 Button
- 3 Resistors
- 1 Breadboard
- 8 Male-to-Male Jumper Wires
- 24 Male-to-Female Jumper Wires
- 1 Potentiometer
- 1 LCD Screen

2. Wiring

- Red LED = GPIO 5
- Green LED = GPIO 26
- Button = GPIO 19
- LCD 4 = GPIO 25
- LCD 6 = GPIO 24
- LCD 11 = GPIO 23
- LCD 12 = GPIO 10
- LCD 13 = GPIO 27
- LCD 14 = GPIO 22

3. How work was divided

What was done	Who did it
Wiring	Me
pin_mode function	Me
digital_write function	Partner
read_button function	Me
Task 1 - Writing Code	Partner
Task 1 - Debugging	Me
Task 2 - Writing Code	Partner
Task 2 - Debugging	Both
Task 3 - Writing Code	Me
Task 3 - Debugging	Me
Task 4 - Writing Code	Me
Task 4 - Debugging	Both
Task 5 - Writing Code	Partner
Task 5 - Debugging	Partner

4. Code Structure

Function	Description
pin_mode(gpio, pin, mode)	Sets 'pin's mode to 'mode' (which is either input or output). This is done using inline ARM assembly in c with pointer arithmetic.
digital_write(gpio, pin, value)	Writes 'value' to 'pin' ('value' is either LOW or HIGH). This is done using inline ARM assembly in C with pointer arithmetic.
read_button(gpio, button)	Reads input from button in pin 'button', which returns 0 for LOW and 1 for HIGH. This is done using inline ARM assembly in C with pointer arithmetic.
hamming(seq1, seq2, length)	Returns the hamming distance of 'seq1' and 'seq2', which is the number of different numbers at each position. This is done using external ARM which is implemented in C with a prototype.

initTimer(timeout)	Timeout mechanism using signals, which is used to determine when a user has not given an input for 'timeout' milliseconds. This is done in C.
submit_PIN(attSeq, seqlen, delay)	Checks if sequence given matches secret sequence by checking if hamming distance of the two sequences is 0. Returns 1 if it is a match and 0 if not. This is done in C.
initSeq(seqlen, digits)	Initiates the secret sequence by Allocating memory if needed and populates with random sequence, ensuring it keeps within boundaries of 'length' and 'digits'. This is done in C.
showSeq(seq, seqlen)	Prints 'seqlen' and 'seq'. This is done in C.
readSeq(seq, seqlen, val)	Converts 'val' (which is an integer) into the format of a sequence (int array). Used for handling command line inputs of sequences which are initially integers (e.g -s 123).
write_LED(gpio, pin, value)	Writes value 'value' to LED in 'pin' (value is either high or low). Used to initialise LEDs to either on (HIGH) or off (LOW). This is done in C.
blinkN(gpio, led, c)	Turns LED in pin 'led' to ON, does a delay and turns LED off, giving a blinking effect. The program executes this sequence 'c' times (e.g if c= 5 then it will blink 5 times). This is done in C.
showHamm(code)	Prints, the hamming distance given.
choosePositions(start, depth, code, positions, checkSeq, attemptSeq, seqlen, digits, submitDelay, opt_e, found, found_at, submits, attempts)	Helper function used in the main to test and generate all sequences at a given hamming distance. Generation is done by genReplacements, which is stored inside this function as a helper. It recursively selects which position in the given sequence will differ from the original sequence. For each valid combination of positions, it resets the working sequence to the original sequence and then calls genReplacements() to test all possible solutions. This is done in C.
genReplacements(start, depth, code, positions, checkSeq, attemptSeq, seqlen, digits, submitDelay, opt_e, found, found_at, submits, attempts)	Helper function for choosePositions(). It assigns all possible alternative values of a selected position in a given sequence, making sure they differ from the original value. Once all selected positions have been assigned new values the generated sequence is submitted for verification. This is done in C.
main()	The core function of the program. It deals with any command line arguments and handles the pin cracking in two general phases (which uses the functions described above to achieve this). Phase 1 involves taking a sequence from user using button presses (if -r is NOT used). Phase 2 involves calculating the hamming distance of the given sequence and the secret sequence, then cracking the pin by enumerating all pins with a matching hamming distance and submitting these pins to find a match. This is done in C.

5. Performance and Resource Consumption

Positive Design decisions:

- The use of two separate functions (choosePositions and genReplacements) opposed to using one function ensures the program only generates sequences for the given hamming distance. This avoids extra computation of all the other hamming distances, therefore improving the runtime, uses less resources and improves efficiency.
- In genReplacements(), the original value in a position is skipped over to the next appropriate number. This avoids generating extra sequences that would not differ at that position and makes sure the program is only generating sequences for the equivalent hamming distance. Therefore, this reduces the number of recursive calls and submissions. In turn, this decision leads to reduced resource consumption and increase in performance and efficiency.

- The arrays used in the program (e.g attemptSeq, foundSeq, etc) are allocated one time and reused throughout the running of the program, opposed to repeated allocation during the recursive calls. After the program completes, the allocated arrays are also freed to ensure proper memory management. This reduces the allocation overhead, reduces memory usage and ensures memory is managed in a sensible manner.
- The use of delay() as opposed to usleep() when debouncing after a button input is a positive decision as usleep can be inaccurate when dealing with inputs that need high precision. This makes the success rate of an accurate button reading greater, leading to a more usable program.

Areas to work on:

- Recursion is used for the generation of sequences. Although this is functional for shorter sequences and lower ranges of digits, the number of recursive calls would cause the program to be inefficient with bigger ranges of digits and lengths. Due to this, stack usage would become far too large and would make the program resource consuming, significantly increasing the runtime, which makes the program less efficient and limits acceptable performance.
- The search space of the sequences grows rapidly as the length/digit range increases, this results in longer runtimes for larger pins with bigger hamming distances, therefore the program can only be considered usable within a certain range.
- With the way the main part of the program is structured, using while loops to check the buttons are being pressed results in the CPU constantly repeatedly checking if the button has been pressed; this wastes CPU cycles and is not productive usage of the CPU, increasing resource consumption.
- In the ARM implementation of pin_mode, the modulo operation is computed manually using UDIV, MUL and SUB. This results in more instructions as it uses a relatively expensive division operation, as opposed to the more efficient % operator built into C, which allws the compiler to optimise the operation. Although this is a minor cost for the computer, this approach is slightly less efficient and introduces unnecessary overhead.

6. Hardware Functions

Below are the list of functions that directly interact with Hardware.

Function	How it interacts with Hardware	Language Used
pin_mode(gpio, pin, mode)	Sets 'pin's mode to 'mode' (which is either input or output).	Inline ARM Assembly in C program and C.
digital_write(gpio, pin, value)	Writes 'value' to 'pin' ('value' is either LOW or HIGH).	Inline ARM Assembly in C program and C.
read_button(gpio, button)	Reads input from button in pin 'button', which returns 0 for LOW and 1 for HIGH.	Inline ARM Assembly in C program and C.
write_LED(gpio, pin, value)	Acts as a wrapper for the digital_write function.	C
blinkN(gpio, led, c)	Uses the write_LED function to make LED light blink.	C
lcd_send4(gpio, value)	Sends four bits of data to LCD display using the digital_write function.	C

lcd_send(gpio, value)	Sends eight bits of data to LCD display by splitting the data into 2 four bits of data and use the lcd_send4 function to handle it.	C
lcd_command(gpio, command)	Sends 'command', which will do what the user specifies to the LCD display.	C
lcd_putchar(gpio, data)	Displays a single character in 'data' on the LCD display. This is done using the digital_write and lcd_send functions.	C
lcd_puts(gpio, data)	Uses the lcd_putchar function on each character in the array 'data', which displays 2 or more characters on the LCD display.	C
lcd_clear(gpio)	Clears display using the lcd_command function to interact with LCD display.	C
lcd_home(gpio)	Returns pointer to top left of LCD display using the lcd_command function to interact with the LCD display.	C
lcd_write_row(gpio, row, data)	Writes characters 'data' on row 'row' on the LCD display. This is done using the lcd_command and lcd_puts functions to interact with the LCD display.	C
lcd_init(gpio)	Initialises the GPIO pins and the LCD display. This is done using the pin_mode, digital_write, lcd_send4 and lcd_command functions.	C

Although most of these functions are written in C, they all rely on the first 3 lower level GPIO functions to interact with hardware. These 3 functions may be ran using the ARM or C code, depending if the ASM flag is enabled.

7. Hamming Function

The hamming function was implemented in ARM assembly in a separate 'hamming.s' file, which followed ARM sub-routine calling conventions (AAPCS). The program begins with an entry point 'global hamming', which allows it to be called in the main C program. The signature for the prototype in C is hamming(const int *x, const int *y, int seqlen), which means the assembly code had to consider 3 inputs from the user. These 3 inputs are automatically stored into r0, r1 and r2 according to the AAPCS calling convention. The beginning of the hamming code pushes r4, r5, r6, r7 and lr as these are callee-saved registers and need to be in the stack to be used. The program runs a loop which repeats the body of the loop until the loop counter reaches the sequence length. This is done so that each number in both sequences can be checked each iteration, until all numbers in both sequences have been compared. The loop then compares the two numbers at the current position, incrementing the hamming distance if the numbers are not equal and continuing to the next number if they are equal. After the hamming distance has been calculated, the output is moved to r0 so the user can obtain the result and the saved registers are restored from the stack.

8. Output Example

sudo ./cw2 -e -n 6 -m 12 -s 123123 -r 111123	sudo ./cw2 -d Raspberry Pi configuration: red LED: 5;
--	--

<pre> Raspberry Pi configuration: red LED: 5; green LED: 26; button: 19 Raspberry Pi LCD driver for a 16x2 display (4-bit wiring) Press ENTER to continue: ----- >> Version 5: arbitrary number of dimensions with 12 digits and 6 sequence length >> HAMM_ASM version: Hamming distance in ARM Assembler ----- Hamming Distance: 2 ----- Runtime; 0.1708 secs Sequence found Exhaustive search finished for 12 digits and 6 seqlen: 1815 attempts (found at 607), 1815 submits ----- Secret sequence was: 1 2 3 1 2 3 Pin found! ----- </pre>	<pre> green LED: 26; button: 19 Raspberry Pi LCD driver for a 16x2 display (4-bit wiring) Press ENTER to continue: ----- >> Version 5: arbitrary number of dimensions with 3 digits and 3 sequence length >> HAMM_ASM version: Hamming distance in ARM Assembler ----- ---- Debug mode ---- The secret sequence is: 3 3 1 The inputted sequence is: 1 3 2 ----- Hamming Distance: 2 ----- Runtime; 0.0009 secs Sequence found Non-exhaustive search finished for 3 digits and 3 seqlen: 7 attempts (found at 7), 7 submits ----- Secret sequence was: 3 3 1 Pin found! ----- </pre>
--	---

9. Summary

This coursework successfully implemented the pinCrack application on a raspberry Pi, combining ARM Assembly and C programming to achieve low level-hardware control. All the tasks given were completed and features such as command line options was successfully implemented to allow configuration and flexible testing. While the more optimised brute force approach of searching for the correct pin works in practice, a more advanced search strategy could be developed in the future to make the pin finding process faster for larger inputs. However, this would be more advanced than the currently implemented searching algorithm.

This process has given a valuable insight into the integration of hardware with software in application, which is something completely new to us. The use of Embedded systems programming has taught us a new level of understanding of GPIO control and the use of inline ARM Assembly is a useful tool for future projects. Using C in this coursework has strengthened our understanding of memory allocation, debugging and programming structure. Throughout the development process, there were several challenges that had to be addressed, such as handling bit offsets, programming correct timers and accurately reading the number of button presses. Through these issues, it has led to a deeper understanding of how systems behave in conjunction with hardware interfaces.

Overall, the coursework provided a practical hands-on experience in combining hardware control with software algorithms and deepened our understanding of how low-level programs differ from standard high-level software development.